

LexMind — AI Legal Assistant Platform

Master Build Specification for Claude Code

Read this entire document before writing any code.

PART 0 — HOW YOU (CLAUDE CODE) SHOULD OPERATE ON THIS PROJECT

1. Work in phases (Part 9), not one giant leap. Complete a phase fully before moving to the next.
 2. Do not stop for permission between phases. Only stop for genuine architectural ambiguity.
 3. Maintain `BUILD_LOG.md` (decisions, shortcuts, what's next) and `ARCHITECTURE.md` (structure, data flow) — update after every phase.
 4. Never silently cut scope. If something is deferred, log it explicitly under "Deferred" with a reason.
 5. After each phase: run it, click through it, check dark mode, check empty/error states, fix before advancing.
 6. Effort management: see Part 11 — this is inserted into your working instructions, not just a note for later.
-

PART 1 — PRODUCT VISION & CRITICAL FRAMING

One-line pitch: "Understand your legal situation in minutes. Built for people — and the lawyers who help them."

What this app IS:

- A legal information, document, and workflow assistant
- A research and drafting accelerator for lawyers

- A plain-language translator for legal complexity

What this app IS NOT — this framing is load-bearing, build it in everywhere:

- Not a substitute for a licensed lawyer’s advice on an actual case
- Not making final legal determinations — it informs, drafts, and organizes; humans decide
- Every consumer-facing legal output must carry a clear, visible disclaimer: this is legal information, not legal advice, and does not create an attorney-client relationship

This isn’t just a legal cover statement — build the product logic around it. E.g., the AI can explain what a “notice of eviction” typically means and draft one, but should encourage the user to have a licensed lawyer (in their jurisdiction) review anything before sending/filing, and should flag when a situation looks complex enough that it needs one.

Global from day one: Jurisdiction is a first-class data field everywhere — not India-only, not US-only. The AI’s context always includes “user’s country/state” and responses are scoped accordingly, with a clear caveat when the AI is uncertain about a specific jurisdiction’s rules.

Two audiences, one platform:

1. **Consumers** — plain-language help, document generation, understanding rights
2. **Lawyers** — research acceleration, drafting assistant, case organization, client intake helper

PART 2 — COMPETITIVE POSITIONING

Existing player	Gap	Our edge
ChatGPT/Gemini (general AI)	No legal-specific workflow, no document generation, no jurisdiction awareness, no case memory	Purpose-built legal workflows + persistent case context
LegalZoom / Rocket Lawyer	Template-heavy, form-filling feel, not conversational, US-centric	AI-conversational, globally scoped, feels like a smart advisor not a form wizard
Lawyer-side	Either practice management OR	Combined research + drafting +

tools (Clio, Harvey AI)	AI research, rarely both, expensive, firm-only	client-facing tool, accessible to solo/small practitioners too
-------------------------	--	--

If a general AI (ChatGPT/Gemini) added “legal mode” tomorrow, would we still have a business? Yes — because the moat is jurisdiction-aware structured workflows, persistent case/document memory, and the dual-sided lawyer+consumer network, not the raw AI capability.

PART 3 — PLATFORM STRATEGY (Web + Desktop + Mobile from One Codebase)

Platform	Approach
Web app	Next.js (React) — primary platform, responsive, this is the core build
Desktop app	Wrap the same Next.js app with Tauri (lighter than Electron, better performance) for Mac/Windows/Linux
Mobile app	Flutter app consuming the same backend/API — separate codebase from web, but shares backend entirely

Why this split: Trying to force one codebase across web+desktop+mobile (e.g., pure React Native Web) usually compromises the quality of all three. Sharing the **backend and design system tokens** across Next.js (web/desktop) and Flutter (mobile) gets you consistency without compromising native feel on mobile.

Backend is platform-agnostic — one API layer (Part 6) serves web, desktop, and mobile identically.

PART 4 — FULL FEATURE SET

4.1 — Shared Foundation (Both User Types)

Onboarding

- Sign up: Email, Google, Apple (Supabase Auth)
- Choose role: “I need legal help” (Consumer) or “I’m a legal professional” (Lawyer) — role determines dashboard, not a hard wall; a consumer can later connect with a real lawyer through the platform

- Jurisdiction setup: country, state/province — stored as structured data, re-askable per case since one user may have matters in multiple jurisdictions
- Language preference (multi-language support from the start — legal help is needed globally, not just in English)

AI Legal Assistant Core (Chat)

- Conversational interface, streaming responses
- Full context: user's jurisdiction, active case/matter, uploaded documents
- Every response scoped with jurisdiction caveats when relevant ("In [state/country], this typically works as... but confirm specifics with a local lawyer since rules vary by [region]")
- Suggested question chips based on common situations (tenant rights, employment disputes, contract questions, small claims, family law basics, etc.)
- Clear, persistent disclaimer banner in every legal chat: "This is legal information, not legal advice."

Document Intelligence

- Upload any legal document (PDF/image/scan) → AI extracts and explains in plain language
- Highlights: key obligations, deadlines, risky clauses, unusual terms
- Side-by-side view: original document + plain-language annotations
- Works for contracts, notices, court documents, agreements — jurisdiction-aware interpretation

Document Generation

- Generate common documents from conversation: demand letters, notices, basic contracts, complaint letters, cease-and-desist, NDAs, etc.
- Template library scoped by jurisdiction and document type
- Every generated document includes a "Review before use" flag and a clear space for a lawyer's review/signature if applicable
- Export as PDF/DOCX

Case/Matter Organization

- Users can create a "Matter" (e.g., "Landlord Dispute," "Employment Contract

Review") that groups: chat history, uploaded documents, generated documents, timeline/deadlines

- Timeline view with deadline tracking and reminders (e.g., "Response due in 14 days")

4.2 — Consumer-Specific Features

- **Situation triage:** guided intake flow — describe your situation in plain language, AI categorizes it (tenant/employment/family/contract/consumer rights/etc.) and suggests next steps
- **Rights explainer:** plain-language breakdown of rights relevant to the user's situation and jurisdiction
- **"Do I need a lawyer?" assessment:** AI gives an honest read on complexity/stakes and recommends whether this is DIY-appropriate or needs professional help
- **Find a lawyer:** directory/marketplace to connect with verified lawyers on the platform for matters that need real representation (this is also the natural monetization bridge to Part 4.3)
- **Cost estimator:** rough, jurisdiction-aware idea of typical costs for common legal actions (filing fees, typical lawyer rates for the matter type) — clearly labeled as estimates

4.3 — Lawyer-Specific Features

- **Research accelerator:** AI-assisted case law and statute research chat, scoped to jurisdiction, with source citations users can verify (never presented as authoritative without sources)
- **Drafting assistant:** AI helps draft/redline contracts, motions, letters — lawyer stays in control, AI produces first drafts and suggestions, not final output
- **Client intake automation:** lawyers can share a branded intake link; consumer's triage conversation (from 4.2) becomes a structured case brief the lawyer receives — saves the lawyer the first consultation's worth of information-gathering
- **Case management lite:** matter list, document storage per client, deadline tracking, notes — not meant to fully replace dedicated practice-management software, but enough for a solo/small practice
- **Client communication log:** organized thread per client/matter, separate from the AI chat, for the lawyer's own notes and client messages

- **Marketplace presence:** lawyer profile (practice areas, jurisdictions licensed in, rate range, reviews) discoverable by consumers via 4.2's "Find a lawyer"

4.4 — Monetization

Tier	Audience	Price	Includes
Free	Consumer	\$0	Limited chat messages/month, 1 active matter, basic document generation
Plus	Consumer	\$14.99/month	Unlimited chat, unlimited matters, full document generation, document intelligence
Professional	Lawyer	\$49.99/month	Full research + drafting tools, client intake automation, case management lite, marketplace profile
Marketplace commission	Both	~15%	Platform takes a cut when a consumer hires a lawyer found through the platform — this is the real scale lever long-term

PART 5 — DESIGN SYSTEM (Premium, Luxury Feel — Non-Negotiable)

5.1 Direction

This needs to feel like a private legal advisor, not a form-filling government website and not a generic SaaS dashboard. Think: the calm authority of a high-end law firm's office, translated into a digital product. Reference feel: Stripe's design polish + a private bank's trust signaling + none of the coldness of typical legal-tech.

5.2 Color Palette

- `ink-navy` #0B1D2E — primary, deep authoritative navy (not black — softer, more premium)
- `gold-accent` #B08D57 — muted brass/gold, used sparingly for premium accents, CTAs, active states (not gaudy bright gold — desaturated, refined)
- `paper-cream` #FAF7F2 — light mode background, evokes quality paper/parchment subtly without being literal

- charcoal-deep #12181F — dark mode background
- text-primary #1A1A1A (light) / #F2F0EC (dark)
- text-muted #6B6B6B (light) / #9A9A9A (dark)
- success-sage #7A9471 — muted, not neon green
- alert-terracotta #C1654A — muted warm red for warnings/deadlines, not alarming pure red
- Full light and dark mode from day one

5.3 Typography

- Headings: A refined serif (e.g., “Fraunces” or “Source Serif 4”) — this is what signals “legal/premium,” used deliberately on key headers and document-related screens
- Body/UI: Clean sans-serif (Inter) for all functional UI text — serif is for gravitas moments, not everywhere
- Clear type scale, generous line-height for legal reading contexts (people will read dense text here — legibility matters more than in a typical consumer app)

5.4 Layout & Motion

- Generous whitespace, structured grid — feels considered, not cluttered
- Border radius: smaller/sharper than a typical playful consumer app (8-12px, not 24px) — reinforces seriousness
- Subtle, restrained motion — fades and gentle slides, not bouncy/playful spring animations. This app should feel composed, not energetic.
- Document-related screens get extra visual care: this is the product’s credibility moment

5.5 Signature Element

The **Document Intelligence** view (original doc + plain-language annotations side by side) is the signature, most-polished screen — this is the “wow” moment equivalent to fridge-scan in the nutrition app. Invest extra design/animation care here: smooth highlight-linking between the original text and its explanation when hovering/tapping.

5.6 Trust & Compliance UI Patterns

- Disclaimer banner component: persistent but not obnoxious, present on every AI

legal output

- Jurisdiction indicator: always visible in the chat/document context so the user knows what legal framework is being applied
- Source citation component: for lawyer-side research features, every claim should be able to show its source

PART 6 — TECH STACK

Layer	Choice	Notes
Web/Desktop framework	Next.js (React) + Tauri for desktop wrapper	Shared codebase for web + desktop
Mobile framework	Flutter	Separate codebase, shared backend
AI (chat + document understanding)	Claude API (Sonnet, vision-capable for document scans)	Core reasoning engine
Backend	Supabase (Postgres, Auth, Storage, Edge Functions)	Shared across all platforms
Auth	Supabase Auth	Email, Google, Apple
Payments	Stripe (web/desktop) + RevenueCat (mobile subscriptions)	Marketplace commission also via Stripe Connect
Document processing	Claude API for extraction/analysis; a PDF library (e.g., pdf-lib) for generation	
Search/citations (lawyer research)	Web search API integration + curated legal database API where available per jurisdiction (defer exact provider selection to a Phase — see Part 9)	

State management	Zustand or Redux Toolkit (web), Riverpod (Flutter)	
------------------	--	--

PART 7 — DATABASE SCHEMA (Core Tables)

```

profiles (
  user_id uuid primary key references auth.users,
  role text, -- 'consumer' | 'lawyer'
  full_name text,
  country text,
  state_province text,
  preferred_language text,
  created_at timestampz default now()
);

lawyer_profiles (
  user_id uuid primary key references profiles(user_id),
  practice_areas jsonb,
  licensed_jurisdictions jsonb,
  bar_number text,
  verification_status text, -- 'pending' | 'verified' | 'rejected'
  rate_range text,
  bio text,
  rating_avg numeric
);

matters (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(user_id),
  title text,
  category text, -- 'tenant' | 'employment' | 'family' | 'contract' | etc.
  jurisdiction_country text,
  jurisdiction_state text,
  status text, -- 'active' | 'resolved' | 'archived'
  assigned_lawyer_id uuid references profiles(user_id),
  created_at timestampz default now()
);

chat_messages (
  id uuid primary key default gen_random_uuid(),
  matter_id uuid references matters(id),
  user_id uuid references profiles(user_id),
  role text, -- 'user' | 'assistant'
  content text,

```

```

    citations jsonb,
    created_at timestamptz default now()
);

documents (
    id uuid primary key default gen_random_uuid(),
    matter_id uuid references matters(id),
    uploaded_by uuid references profiles(user_id),
    type text, -- 'uploaded' | 'generated'
    file_url text,
    extracted_text text,
    ai_annotations jsonb,
    created_at timestamptz default now()
);

deadlines (
    id uuid primary key default gen_random_uuid(),
    matter_id uuid references matters(id),
    title text,
    due_date date,
    status text, -- 'upcoming' | 'completed' | 'missed'
    reminder_sent boolean default false
);

lawyer_client_links (
    lawyer_id uuid references profiles(user_id),
    client_id uuid references profiles(user_id),
    matter_id uuid references matters(id),
    status text, -- 'invited' | 'active' | 'ended'
    primary key (lawyer_id, client_id, matter_id)
);

subscriptions (
    user_id uuid primary key references profiles(user_id),
    tier text, -- 'free' | 'plus' | 'professional'
    status text,
    renews_at timestamptz
);

```

Apply row-level security on every table. Lawyer-client data isolation is critical here — a lawyer should only see matters/documents explicitly shared with them via `lawyer_client_links`, never all consumer data.

PART 8 — CLAUDE API INTEGRATION

Consumer Chat System Prompt (Base)

You are a legal information assistant inside LexMind. You are NOT a lawyer and this is NOT legal advice – you provide legal information and help the user understand their situation and options.

User context:

- Jurisdiction: {country}, {state_province}
- Active matter: {matter_category}, {matter_title}
- Matter history: {matter_summary}

Guidelines:

- Explain legal concepts in plain, accessible language
- Always scope your answer to the user's jurisdiction; if you're not certain how a specific rule applies in their jurisdiction, say so clearly and recommend confirming with a local lawyer
- If the situation involves high stakes (criminal matters, significant money, family custody, immigration status), clearly recommend professional legal representation, don't just answer and move on
- Never state something as certain legal fact if it varies significantly by jurisdiction – flag the variation
- End complex responses with a natural next-step suggestion (draft a document, find a lawyer, or a follow-up question)

Lawyer Research System Prompt (Base)

You are a legal research assistant for licensed legal professionals inside LexMind. The user is a verified lawyer – you can use more technical legal language than with consumer users, but citation discipline matters more, not less.

Jurisdiction: {jurisdiction}

Matter context: {matter_summary}

Guidelines:

- Cite sources (statutes, cases, regulations) for substantive claims wherever possible
- Distinguish clearly between settled law and areas of genuine legal uncertainty/circuit split/jurisdictional variation
- When drafting, produce a first draft the lawyer can edit – flag areas that need the lawyer's judgment call rather than guessing
- You are a research accelerator, not a replacement for the lawyer's own verification – never present yourself as the final authority

Document Analysis — Structured Output

```
{
  "document_type": "residential lease agreement",
  "jurisdiction_relevant": true,
  "key_obligations": [
    {"party": "tenant", "obligation": "...", "deadline_if_any": "..."}
  ],
  "risky_or_unusual_clauses": [
    {"clause_excerpt_summary": "...", "why_flagged": "...", "section_reference": ...}
  ],
  "important_dates": [...],
  "plain_language_summary": "..."
}
```

Error Handling

Never expose raw API errors. On failure, show a calm, professional retry message consistent with the app's tone — never casual/playful language here, this app's error states should still feel composed.

PART 9 — BUILD ORDER (Phased, Sequential)

Phase 1 — Foundation & Compliance Framework Next.js project scaffold, design system as theme tokens (Part 5), Supabase setup, database schema (Part 7) with RLS policies, disclaimer/trust UI components (built once, reused everywhere — this is core to the product, not an afterthought).

Phase 2 — Auth & Onboarding Auth flow, role selection (consumer/lawyer), jurisdiction setup, language preference, profile creation for both roles.

Phase 3 — Core Chat (Consumer) Chat UI, streaming responses, Claude API integration with consumer system prompt, jurisdiction context injection, suggested question chips, persistent disclaimer component.

Phase 4 — Matters & Case Organization Matter creation/management, timeline/deadline UI, linking chat history to matters.

Phase 5 — Document Intelligence (Signature Feature — Extra Polish) Upload flow, Claude Vision extraction, structured annotation output, the side-by-side original+annotation UI with highlight-linking interaction (Part 5.5).

Phase 6 — Document Generation Template library structure, conversation-to-

document flow, PDF/DOCX export, “review before use” flagging.

Phase 7 — Lawyer Research & Drafting Tools Lawyer-specific chat mode with research system prompt, citation display component, drafting assistant flow.

Phase 8 — Client Intake & Case Management (Lawyer Side) Intake link generation, triage-to-brief conversion, lawyer’s client/matter dashboard, communication log.

Phase 9 — Lawyer Marketplace Lawyer profile/verification flow, consumer-facing “Find a lawyer” directory and search, lawyer-client linking flow.

Phase 10 — Monetization Stripe integration (web/desktop), RevenueCat (mobile), tier gating across all features, Stripe Connect for marketplace commission.

Phase 11 — Desktop Wrapper Tauri packaging of the Next.js app for Mac/Windows/Linux, native menu/window polish.

Phase 12 — Mobile App (Flutter) Separate Flutter build consuming the same backend — core chat, matters, document intelligence, document generation ported to mobile-native UI following the same design tokens.

Phase 13 — Polish Loop (Repeat Until Clean) Full click-through on web, desktop, and mobile. Dark mode everywhere. Empty/error states. Animation smoothness. Fix and repeat.

PART 10 — QUALITY BARS

- 60fps on all web/desktop animations, no jank
 - Document analysis returns in under 5-6 seconds including API round trip, with an engaging progress state
 - Full WCAG AA accessibility — this matters more here than average, legal help must be accessible
 - Every AI legal output carries the disclaimer — this is a hard requirement, not a nice-to-have
 - Row-level security tested explicitly — a lawyer must never be able to query another lawyer’s clients’ data
 - 100% dark/light mode parity across all three platforms
-

PART 11 — EFFORT MANAGEMENT FOR CLAUDE CODE

Running at MAX effort is expensive — reserve it for phases where architecture mistakes are costly or legal-compliance logic needs extra care:

Phase	Effort	Why
Phase 1 (Foundation & Compliance)	MAX	RLS policies + compliance UI patterns are foundational, mistakes propagate everywhere
Phase 2 (Auth & Onboarding)	HIGH	Standard patterns
Phase 3 (Core Chat)	HIGH	Standard chat UI, prompt already specified
Phase 4 (Matters)	HIGH	Standard CRUD
Phase 5 (Document Intelligence)	MAX	Signature feature, structured extraction accuracy matters most here
Phase 6 (Document Generation)	MAX	Legal document accuracy/formatting has real consequences
Phase 7 (Lawyer Research)	MAX	Citation discipline and legal reasoning quality are the core value prop for paying lawyers
Phase 8 (Client Intake)	HIGH	Standard workflow
Phase 9 (Marketplace)	HIGH	Standard directory/matching logic
Phase 10 (Monetization)	MAX	Payment/commission bugs are costly to debug later
Phase 11 (Desktop Wrapper)	HIGH	Mostly packaging, not novel logic
Phase 12 (Mobile)	HIGH	Porting established patterns to Flutter
Phase 13 (Polish Loop)	HIGH	Iterative fixing

PART 12 — THE PROMPT TO PASTE INTO CLAUDE CODE

Save this entire document as `LexMind_MasterSpec.md` in your project root first. Then paste:

I'm building LexMind – a premium legal information and workflow platform, serving both consumers and lawyers, across web, desktop, and mobile. The full specification is saved as `LexMind_MasterSpec.md` in this project root. Read that entire document now before writing any code.

Operate as an autonomous build loop per Part 0: work through the phases in Part 9 sequentially, don't stop for permission between phases, maintain `BUILD_LOG.md` and `ARCHITECTURE.md`, and only surface a question for genuine architectural ambiguity.

Build the FULL feature set in Part 4 – consumer AND lawyer sides, do not silently cut scope. Log any deferral explicitly in `BUILD_LOG.md` with a reason.

CRITICAL – Part 1's framing (legal information, not legal advice) and Part 10's compliance quality bars are non-negotiable. Every AI legal output needs the disclaimer pattern from Phase 1 built in and reused everywhere, not bolted on later.

Design quality bar: Part 5 is non-negotiable – premium, composed, trustworthy, not a generic SaaS template. Document Intelligence (Phase 5) is the signature feature, invest extra polish there.

EFFORT MANAGEMENT – I'm running you at MAX effort right now. Per Part 11, only these phases need max:

- Phase 1 – Foundation & Compliance
- Phase 5 – Document Intelligence
- Phase 6 – Document Generation
- Phase 7 – Lawyer Research & Drafting
- Phase 10 – Monetization

At the START of each phase, print:

```
">>> STARTING PHASE X – recommended effort: [MAX/HIGH] – set this now  
with /effort if needed, then confirm and continue."
```

Pause there for my confirmation only – not for approval of the work itself.

Start now with Phase 1 at MAX effort: project scaffold, design system as theme tokens, Supabase setup, full database schema with RLS policies, and the reusable disclaimer/trust UI components. Verify RLS policies actually work (test that one user cannot query another's data) before marking this phase complete. Update `BUILD_LOG.md` and `ARCHITECTURE.md`.

NOTES FOR YOU (Not Part of the Claude Code Prompt)

- **This is a bigger build than NutriMind** — two user types, three platforms, and real compliance considerations. Budget realistically for time, not just enthusiasm.
- **Lawyer verification (Phase 9) is a real-world process, not just a UI toggle** — you'll eventually need an actual process for confirming someone is a licensed lawyer (bar number checks vary by country). Claude Code can build the UI/data structure; the actual verification workflow needs your judgment call on how rigorous to be at launch.
- **Consider starting with ONE jurisdiction well** (e.g., your home country) even though the architecture supports global from day one — "globally scoped architecture, launch focused" is very different from "trying to be correct about every country's law on day one." The data model supports expansion; you don't have to fill it all in at once.
- **Legal-tech has real compliance considerations that vary by country** (unauthorized practice of law rules differ everywhere) — this spec is built to minimize risk by staying in "information + drafting assistance," but you may want a lawyer's actual review of your terms of service before public launch, not just for the product content.
- **Budget:** Domain/hosting, Supabase, Claude API (likely higher usage than NutriMind given longer documents/conversations — budget more like \$50-150/month during testing), Stripe fees, Apple/Google developer accounts for the mobile app.